Documentation Technique — AELI SERVICES

> **Version :** 1.0
> **Date :** Mars 2026
> **Auteur :** NGOUE DAVID
> **Statut :** Production

INTRODUCTION

Présentation du projet

AELI Services est une plateforme de mise en relation entre clients et prestataires de services féminins au Cameroun. Elle permet aux utilisateurs de rechercher, contacter et évaluer des prestataires, et offre à ces dernières un espace de gestion de leur offre, de leur visibilité et de leur facturation. La plateforme repose sur trois applications distinctes et complémentaires : une API backend centralisée, une interface client/prestataire et un tableau de bord d'administration.

Objectifs de la documentation

Cette documentation a pour but de fournir une référence technique exhaustive couvrant l'installation, la configuration, les contrats d'interface (API), la logique métier, ainsi que les guides d'intégration frontend — aussi bien pour l'espace client/prestataire que pour l'interface d'administration. Elle constitue la source de vérité technique pour tout développeur ou administrateur système intervenant sur le projet.

Public visé

- **Développeurs backend** : installation, configuration de l'environnement, référence des endpoints, logique métier et sécurité.
- **Développeurs frontend** : intégration du client HTTP, gestion d'état avec React Query, implémentation des composants, déploiement et optimisation.
- **Administrateurs système** : déploiement Docker, gestion des variables d'environnement, monitoring des logs.

MODULE 1 : BACKEND & INFRASTRUCTURE (CORE API)

1. Architecture & Environnement

1.1. Schéma de l'infrastructure

L'infrastructure backend repose sur trois composants principaux orchestrés via Docker :

```
┌──────────────────────────────────────────────────────────────────────┐
│                Docker Network                                        │
│                                                                      │
│  ┌──────────────┐   ┌──────────────┐   ┌──────────────┐            │
│  │   Nginx      │──▶│ Express.js   │──▶│ PostgreSQL   │            │
│  │ (Reverse     │   │ API :5000    │   │ :5432        │            │
│  │  Proxy)      │   │              │   │              │            │
│  └──────────────┘   └──────┬───────┘   └──────────────┘            │
│                            │                                         │
│                            ▼                                         │
│                     ┌──────────────┐                                 │
│                     │   Redis      │                                 │
│                     │ (Queue &     │                                 │
│                     │  Cache)      │                                 │
│                     └──────────────┘                                 │
└──────────────────────────────────────────────────────────────────────┘
```

Rôle de chaque composant :

- **Express.js (Node.js 18+)** : Cœur de l'API REST. Gère le routage, les middlewares d'authentification, la validation des requêtes et l'orchestration de la logique métier via 11 contrôleurs.
- **PostgreSQL** : Base de données relationnelle principale. L'accès est abstrait par **Sequelize ORM** qui gère 13 modèles avec leurs hooks, relations et validations.
- **Redis** : Couche de cache et file de traitement asynchrone des tâches (jobs Bull) telles que l'envoi différé d'emails ou le renouvellement automatique des abonnements.
- **Nginx** : Reverse proxy en production pour la terminaison SSL, la distribution du trafic et la mise en cache des ressources statiques.
- **Cloudinary** : Service externe pour le stockage et la transformation des images (photos de profil prestataire, bannières).

1.2. Configuration Système

Toutes les configurations sensibles sont externalisées dans un fichier `.env` à la racine du projet. Le fichier `.env.example` fourni dans le dépôt sert de modèle.

Variables d'environnement obligatoires :

Variable	Description	Exemple
`NODE_ENV`	Environnement d'exécution	`production`
`PORT`	Port d'écoute du serveur	`5000`
`CORS_ORIGIN`	Origine(s) autorisées pour CORS	`https://app.aeli.cm`
`DB_HOST`	Hôte PostgreSQL	`localhost`
`DB_PORT`	Port PostgreSQL	`5432`
`DB_NAME`	Nom de la base de données	`aeli_services`
`DB_USER`	Utilisateur PostgreSQL	`aeli_user`
`DB_PASSWORD`	Mot de passe PostgreSQL	`***`
`JWT_SECRET`	Clé secrète de signature des tokens d'accès	`<chaîne aléatoire 256 bits>`
`JWT_EXPIRE`	Durée de validité du token d'accès	`7d`
`JWT_REFRESH_EXPIRE`	Durée de validité du refresh token	`30d`
`CLOUDINARY_CLOUD_NAME`	Nom du cloud Cloudinary	`aeli-cloud`
`CLOUDINARY_API_KEY`	Clé API Cloudinary	`***`
`CLOUDINARY_API_SECRET`	Secret API Cloudinary	`***`
`EMAIL_HOST`	Hôte SMTP	`smtp.mailtrap.io`
`EMAIL_PORT`	Port SMTP	`2525`
`EMAIL_USER`	Utilisateur SMTP	`***`
`EMAIL_PASS`	Mot de passe SMTP	`***`
`REDIS_HOST`	Hôte Redis	`localhost`
`REDIS_PORT`	Port Redis	`6379`
`CINETPAY_API_KEY`	Clé API CinetPay	`***`
`NOTCHPAY_API_KEY`	Clé API NotchPay	`***`

> ⚠️ **Sécurité** : Ne jamais committer le fichier `.env` dans le dépôt. Utiliser un gestionnaire de secrets (ex: Vault, AWS Secrets Manager) en production.

1.3. Guide d'Installation

Prérequis

- Node.js 18+
- PostgreSQL 14+
- Redis 6+
- Docker & Docker Compose (pour le déploiement conteneurisé)

Installation locale (développement)

Étape 1 — Cloner et installer les dépendances

```bash
git clone https://github.com/AELISERVICE/AELI-SERVICE-.git
cd AELI-SERVICE-
npm install
```

Étape 2 — Configurer les variables d'environnement

```bash
cp .env.example .env
# Éditer .env avec vos valeurs
```

Étape 3 — Créer la base de données PostgreSQL

```sql
CREATE DATABASE aeli_services;
```

Étape 4 — Exécuter les migrations Sequelize

```bash
# Appliquer toutes les migrations en attente
npx sequelize-cli db:migrate

# Injecter les données de test (optionnel)
npx sequelize-cli db:seed:all
```

Gestion des migrations avec Sequelize CLI :

```bash
```

```
# Créer une nouvelle migration
npx sequelize-cli migration:generate --name nom-de-la-migration

# Annuler la dernière migration
npx sequelize-cli db:migrate:undo

# Annuler toutes les migrations
npx sequelize-cli db:migrate:undo:all
```

Étape 5 — Démarrer le serveur

```bash
# Mode développement (avec rechargement automatique via nodemon)
npm run dev

# Mode production
npm start
```

L'API sera accessible sur `http://localhost:5000`.

Déploiement via Docker Compose

```bash
# Construire les images
npm run docker:build
# ou : docker-compose build

# Démarrer tous les services (API, PostgreSQL, Redis, Nginx)
npm run docker:up
# ou : docker-compose up -d

# Consulter les logs en temps réel
npm run docker:logs
# ou : docker-compose logs -f

# Arrêter les services
npm run docker:down
# ou : docker-compose down
```

Vérification de l'état du service :

```bash
```

```
# Santé globale de l'API
curl http://localhost:5000/api/health

# Connectivité à la base de données
curl http://localhost:5000/api/health/db
```

---

2. Référence de l'API (Standard OpenAPI)

2.1. Sécurité & Authentification

Flux OAuth2 / JWT

AELI Services implémente un flux d'authentification par **double token JWT** (Access Token + Refresh Token) combiné à une vérification OTP par email.

```
┌─────────┐      ┌─────────────┐      ┌─────────────┐
│ Client │      │ API    │      │ Email  │
└────┬────┘      └──────┬──────┘      └──────┬──────┘
   │ POST /register    │              │
   │──────────────────────────────▶│              │
   │              │── Envoi OTP ──────▶│
   │◀── 201 (pending)──│              │
   │              │              │
   │ POST /verify-otp  │              │
   │──────────────────────────────▶│              │
   │◀── accessToken    │              │
   │   refreshToken   │              │
   │              │              │
   │ GET /protected    │              │
   │ Authorization: Bearer <accessToken>   │
   │──────────────────────────────▶│              │
   │◀── 200 data ──────│              │
   │              │              │
   │ (token expiré)    │              │
   │ POST /auth/refresh │              │
   │ refreshToken      │              │
   │──────────────────────────────▶│              │
   │◀── nouveau access │              │
   │   Token        │              │
└────┴────┘      └──────┴──────┘      └──────┴──────┘
```

Utilisation du token

Toutes les routes protégées requièrent l'en-tête suivant :

```
Authorization: Bearer <accessToken>
```

Gestion des rôles

Rôle	Description	Accès
`user`	Client standard de la plateforme	Routes publiques + profil propre
`provider`	Prestataire validé	Routes `user` + gestion de son offre + dashboard
`admin`	Administrateur de la plateforme	Accès total à toutes les routes `/api/admin/*`

Les routes admin vérifient le rôle via le middleware `requireAdmin`. Les routes prestataires vérifient le statut via `requireProvider`.

Endpoints d'authentification

Méthode	Endpoint	Auth	Description
`POST`	`/api/auth/register`	✗	Inscription + envoi OTP
`POST`	`/api/auth/verify-otp`	✗	Vérification OTP → retourne les tokens
`POST`	`/api/auth/resend-otp`	✗	Renvoi de l'OTP
`POST`	`/api/auth/login`	✗	Connexion → retourne les tokens
`POST`	`/api/auth/refresh`	✗	Renouvellement du token d'accès
`POST`	`/api/auth/logout`	✓	Invalidation de la session
`POST`	`/api/auth/forgot-password`	✗	Demande de réinitialisation du mot de passe
`POST`	`/api/auth/reset-password/:token`	✗	Réinitialisation avec token email
`GET`	`/api/auth/me`	✓	Profil de l'utilisateur connecté

2.2. Catalogue des Points d'Entrée (Endpoints)

Gestion des Utilisateurs (`/api/users`)

Méthode	Endpoint	Auth	Rôle	Description
GET	/api/users/profile	✓	user	Récupérer son profil
PUT	/api/users/profile	✓	user	Mettre à jour son profil
PUT	/api/users/password	✓	user	Changer son mot de passe
DELETE	/api/users/account	✓	user	Désactiver son compte

Gestion des Prestataires (`/api/providers`)

Méthode	Endpoint	Auth	Rôle	Description
GET	/api/providers	✗	—	Liste paginée et filtrable des prestataires
GET	/api/providers/:id	✗	—	Détail d'un prestataire
POST	/api/providers/apply	✓	user	Soumettre une candidature prestataire
GET	/api/providers/my-application	✓	user	Statut de sa propre candidature
PUT	/api/providers/:id	✓	provider	Modifier son profil prestataire
DELETE	/api/providers/:id/photos/:index	✓	provider	Supprimer une photo de profil
GET	/api/providers/my-profile	✓	provider	Récupérer son profil prestataire
GET	/api/providers/my-dashboard	✓	provider	Tableau de bord prestataire (stats)

Paramètres de recherche pour `GET /api/providers` :

Paramètre	Type	Description
`q`	string	Terme de recherche (nom, service)
`category`	string	Slug de catégorie
`location`	string	Ville ou quartier
`minRating`	number	Note minimale (1–5)
`maxPrice`	number	Prix maximum
`sort`	string	Tri : `rating`, `recent`, `views`
`page`	integer	Numéro de page (défaut : 1)
`limit`	integer	Éléments par page (défaut : 20)

Gestion des Services (`/api/services`)

Méthode	Endpoint	Auth	Rôle	Description
`GET`	`/api/services/categories`	✗	—	Liste des catégories de services
`POST`	`/api/services/categories`	✓	admin	Créer une catégorie
`GET`	`/api/services/provider/:id`	✗	—	Services d'un prestataire donné
`POST`	`/api/services`	✓	provider	Créer un service
`PUT`	`/api/services/:id`	✓	provider	Modifier un service
`DELETE`	`/api/services/:id`	✓	provider	Supprimer un service

Marketplace — Mise en relation (`/api/contacts`)

Méthode	Endpoint	Auth	Rôle	Description
POST	/api/contacts	✓	user	Envoyer un message à un prestataire
GET	/api/contacts/received	✓	provider	Messages reçus (côté prestataire)
GET	/api/contacts/sent	✓	user	Messages envoyés
PUT	/api/contacts/:id/status	✓	provider	Mettre à jour le statut d'un message
POST	/api/contacts/:id/unlock	✓	user	Débloquer un message (paiement ou abonnement)
GET	/api/contacts/stats/daily	✓	provider	Statistiques journalières de contact

Avis & Notation (`/api/reviews`)

Méthode	Endpoint	Auth	Rôle	Description
GET	/api/reviews/provider/:id	✗	—	Avis d'un prestataire
POST	/api/reviews	✓	user	Créer un avis
PUT	/api/reviews/:id	✓	user	Modifier son avis
DELETE	/api/reviews/:id	✓	user	Supprimer son avis

Favoris (`/api/favorites`)

Méthode	Endpoint	Auth	Rôle	Description
GET	/api/favorites	✓	user	Liste de ses favoris
POST	/api/favorites	✓	user	Ajouter un favori
DELETE	/api/favorites/:providerId	✓	user	Supprimer un favori
GET	/api/favorites/check/:providerId	✓	user	Vérifier si un prestataire est en favori

Paiements & Abonnements (`/api/subscriptions` & `/api/payments`)

Méthode	Endpoint	Auth	Rôle	Description
GET	/api/subscriptions/plans	✗	—	Liste des plans disponibles
GET	/api/subscriptions/my	✓	provider	Abonnement actif du prestataire connecté
POST	/api/subscriptions/subscribe	✓	provider	Initier un paiement d'abonnement (NotchPay)

Administration (/api/admin)

Méthode	Endpoint	Auth	Rôle	Description
GET	/api/admin/stats	✓	admin	Statistiques globales de la plateforme
GET	/api/admin/users	✓	admin	Liste de tous les utilisateurs
PUT	/api/admin/users/:id/status	✓	admin	Activer / désactiver un compte utilisateur
GET	/api/admin/providers/pending	✓	admin	Candidatures prestataires en attente
PUT	/api/admin/providers/:id/verify	✓	admin	Valider ou rejeter une candidature
PUT	/api/admin/providers/:id/feature	✓	admin	Mettre en avant un prestataire
PUT	/api/admin/providers/:id/status	✓	admin	Activer / désactiver un compte prestataire
GET	/api/admin/reviews	✓	admin	Tous les avis de la plateforme
PUT	/api/admin/reviews/:id/visibility	✓	admin	Masquer / afficher un avis
DELETE	/api/admin/reviews/:id	✓	admin	Supprimer un avis
GET	/api/admin/payments	✓	admin	Liste des paiements
GET	/api/admin/security/stats	✓	admin	Statistiques de sécurité
GET	/api/admin/security/logs	✓	admin	Logs de sécurité et IPs à risque
GET	/api/admin/security/banned-ips	✓	admin	Liste des IPs bannies
DELETE	/api/admin/security/banned-ips/:ip	✓	admin	Débannir une IP
GET	/api/admin/export/users	✓	admin	Export CSV des utilisateurs
GET	/api/admin/export/providers	✓	admin	Export CSV des prestataires
GET	/api/admin/export/reviews	✓	admin	Export CSV des avis
GET	/api/admin/export/global	✓	admin	Rapport global (PDF/CSV)

Bannières marketing (`/api/banners`)

Méthode	Endpoint	Auth	Rôle	Description
GET	/api/banners	✓	admin	Liste des bannières (admin)
GET	/api/banners/public	✗	—	Bannières publiques actives
POST	/api/banners	✓	admin	Créer une bannière (avec image)
PUT	/api/banners/:id	✓	admin	Modifier une bannière
DELETE	/api/banners/:id	✓	admin	Supprimer une bannière

2.3. Contrats d'Interface (Modèles de données)

Modèle `User` (Requête — Inscription)

```json
{
  "firstName": "string (requis)",
  "lastName": "string (requis)",
  "email": "string (email valide, requis)",
  "password": "string (min. 8 caractères, requis)",
  "phone": "string (optionnel)",
  "language": "string (fr | en, défaut: fr)"
}
```

Modèle `User` (Réponse)

```json
{
  "id": "uuid",
  "firstName": "string",
  "lastName": "string",
  "email": "string (chiffré en BDD, déchiffré à la lecture)",
  "phone": "string",
  "role": "user | provider | admin",
  "isActive": "boolean",
  "createdAt": "ISO 8601"
}
```

Modèle `AuthTokens` (Réponse après login/OTP)

```json
{
  "accessToken": "string (JWT, expiration configurée via JWT_EXPIRE)",
  "refreshToken": "string (JWT longue durée, expiration via JWT_REFRESH_EXPIRE)",
  "user": { "...champs User..." }
}
```

Modèle `Provider` (Réponse)

```json
{
  "id": "uuid",
  "userId": "uuid",
  "businessName": "string",
  "description": "string",
  "location": "string",
  "latitude": "float",
  "longitude": "float",
  "photos": ["url1", "url2", "..."],
  "rating": "float (0–5)",
  "reviewCount": "integer",
  "isVerified": "boolean",
  "isFeatured": "boolean",
  "subscriptionStatus": "active | expired | none",
  "services": [{ "...champs Service..." }]
}
```

Modèle `Service` (Requête — Création)

```json
{
  "categoryId": "uuid (requis)",
  "title": "string (requis)",
  "description": "string (requis)",
  "price": "number (requis)",
  "duration": "string (ex: '2h')",
  "image": "File (FormData, optionnel)"
}
```

Modèle `Review` (Requête — Création)

```json
```

```
{
  "providerId": "uuid (requis)",
  "rating": "integer (1–5, requis)",
  "comment": "string (requis)"
}
```

Format de réponse d'erreur standard

```
json

{
  "success": false,
  "code": "ERROR_CODE",
  "message": "Message lisible par l'humain (localisé fr/en)",
  "details": {}
}
```

---

3. Logique Métier & Sécurité

3.1. Workflows Critiques

Workflow de candidature prestataire

```
Client        POST /providers/apply        Admin
  │──────────────────────────────────────────────────────►│
  │          (Statut: "pending")          │
  │                                       │
  │            (Email notification)       │
  │◄──────────────────────────────────────────────────────│
  │                                       │
  │                PUT /admin/providers/:id/verify
  │                { status: "approved" } │
  │◄──────────────────────────────(Email confirmation)────│
  │   (Rôle promu à "provider")           │
```

1. L'utilisateur envoie sa candidature avec les documents requis.
2. La candidature est enregistrée avec le statut `pending`.
3. Un email de notification est envoyé à l'équipe admin.
4. L'administrateur consulte la candidature et ses documents dans le back-office.
5. En cas d'approbation, le rôle de l'utilisateur est mis à jour vers `provider` et un email de confirmation lui est envoyé.

6. En cas de rejet, un email motivé lui est envoyé.

Workflow de déverrouillage d'un contact (pay-per-view)

1. Un client envoie un message à un prestataire via `POST /contacts`.
2. Le message est chiffré et stocké avec le statut `locked`.
3. Le prestataire ne peut lire le contenu qu'après paiement ou s'il dispose d'un abonnement actif.
4. L'appel `POST /contacts/:id/unlock` initie le paiement via CinetPay/NotchPay.
5. Après confirmation du paiement (webhook entrant), le statut passe à `unlocked`.

Workflow de renouvellement automatique des abonnements

Un job Bull planifié s'exécute quotidiennement et vérifie les abonnements arrivant à expiration. Pour chaque abonnement éligible à l'auto-renouvellement, il initie une nouvelle transaction de paiement. En cas d'échec, il décrémente un compteur de tentatives et notifie le prestataire par email.

3.2. Gestion des Erreurs & Logs

Inventaire des codes d'erreur métier

Code	HTTP	Description
`TOKEN_EXPIRED`	401	Le token d'accès JWT a expiré
`TOKEN_INVALID`	401	Token malformé ou signature invalide
`UNAUTHORIZED`	403	Rôle insuffisant pour cette ressource
`USER_NOT_FOUND`	404	Utilisateur introuvable
`PROVIDER_NOT_FOUND`	404	Prestataire introuvable
`OTP_INVALID`	400	Code OTP incorrect
`OTP_EXPIRED`	400	Code OTP expiré
`EMAIL_ALREADY_EXISTS`	409	Email déjà utilisé
`REVIEW_ALREADY_EXISTS`	409	L'utilisateur a déjà laissé un avis pour ce prestataire
`CONTACT_LOCKED`	402	Message verrouillé, paiement requis
`INSUFFICIENT_SUBSCRIPTION`	402	Abonnement inactif ou expiré
`RATE_LIMIT_EXCEEDED`	429	Trop de requêtes (rate limiting)
`VALIDATION_ERROR`	422	Données de requête invalides
`INTERNAL_ERROR`	500	Erreur serveur non gérée

Monitoring des logs via Winston

Les logs sont structurés au format JSON et répartis en deux fichiers :

- **`logs/error.log`** : Erreurs uniquement (niveau `error`). Contient les stack traces, les codes d'erreur métier et les détails de contexte (userId, endpoint, IP).
- **`logs/combined.log`** : Tous les niveaux (`info`, `warn`, `error`). Contient les logs d'audit des actions sensibles (connexions, modifications de rôle, paiements).

Exemple de log structuré :

```
json
```

```
{
  "level": "error",
  "timestamp": "2026-03-07T10:42:00.000Z",
  "message": "Authentication failed",
  "code": "TOKEN_INVALID",
  "userId": null,
  "ip": "196.202.xxx.xxx",
  "endpoint": "GET /api/providers/my-profile",
  "requestId": "req-abc123"
}
```

Pour la supervision en production, il est recommandé d'utiliser **PM2** couplé à un agrégateur de logs (ex: Datadog, Logtail ou ELK Stack) pour monitorer les fichiers de logs en temps réel et configurer des alertes sur les patterns d'erreurs critiques.

Sécurité de l'infrastructure

Chiffrement des données sensibles : Les emails et numéros de téléphone des utilisateurs sont chiffrés en base de données via AES-256 et déchiffrés à la lecture par les méthodes d'instance Sequelize. Les mots de passe sont hachés avec bcrypt (salt rounds configurables).

Rate limiting : Configuré par endpoint — 5 tentatives de connexion par 15 minutes par IP sur `/api/auth/login`, 100 requêtes par minute en général.

Protection des attaques : Helmet.js applique des en-têtes HTTP de sécurité stricts (CSP, HSTS, X-Frame-Options). La validation des entrées via des validators dédiés protège contre les injections. Sequelize ORM utilise des requêtes paramétrées contre les injections SQL.

MODULE 2 : INTÉGRATION FRONTEND (ADMIN & CLIENT)

1. Configuration du Client API

Les deux interfaces frontend (client/prestataire et admin) partagent la même architecture de communication, centralisée dans `src/api/apiClient.jsx`.

Stack commune

Technologie	Version	Rôle
React	19	Construction de l'interface
React Router	7	Routage SPA
TanStack React Query	5	Cache et synchronisation des données

Technologie	Version	Rôle
Tailwind CSS	4	Stylisation utilitaire
React Toastify	—	Notifications
Vite	7 (Rolldown)	Bundler et serveur de développement
ESLint	9	Qualité du code
Nginx	—	Serveur de production

1.1. Logique du Client HTTP (`apiClient.jsx`)

Le client centralise toutes les interactions réseau en un wrapper autour de l'API native `fetch`.

Gestion des en-têtes : Le client injecte automatiquement le header `Authorization: Bearer <token>` pour chaque requête en lisant le token depuis le `localStorage`. Pour les requêtes incluant un `FormData` (upload de fichiers), le header `Content-Type` est omis intentionnellement afin que le navigateur le génère avec le bon `boundary` multipart. Pour toutes les autres requêtes, il est fixé à `application/json`.

Mécanisme de Silent Refresh : Ce mécanisme garantit une session transparente pour l'utilisateur. Le cycle de vie est le suivant :

1. Une requête protégée est émise avec l'`accessToken` courant.
2. Si le serveur répond avec le code `TOKEN_EXPIRED`, le client intercepte l'erreur avant de la propager.
3. Il appelle automatiquement `POST /api/auth/refresh` avec le `refreshToken` stocké en `localStorage`.
4. Si le refresh réussit, le nouveau `accessToken` est sauvegardé et la requête initiale est rejouée automatiquement, de manière totalement transparente pour l'utilisateur.
5. Si le refresh échoue (token révoqué ou expiré), la session est purgée (`localStorage.clear()`) et l'utilisateur est redirigé vers `/login`.

Sécurisation du parsing : Un helper interne `getSafeJson` protège le client contre les crashs lors du parsing de réponses vides (204 No Content) ou de pages d'erreur HTML retournées par le serveur en cas de problème infrastructure.

1.2. Gestion d'État Global avec React Query

TanStack React Query 5 est utilisé comme couche de gestion des données asynchrones. Il découple la logique de récupération de la logique d'affichage et fournit un cache automatique par clé de requête.

Cycle de vie des états exposés par les hooks :

État	Description	Impact sur l'interface
`isLoading`	Chargement initial (pas encore de données en cache)	Affichage des Skeletons (`React Loading Skeleton`)
`isPending`	Mutation en cours (création, modification, suppression)	Désactivation des boutons, affichage de loaders circulaires
`isSuccess`	Données reçues avec succès	Rendu des composants (tableaux, graphiques Recharts)
`isError`	Échec de la communication avec l'API	Déclenchement d'un toast `React Toastify` avec le message d'erreur
`data`	Objet de réponse JSON	Distribution des données vers les composants fils via props
`error`	Détails de l'exception levée	Log technique ou message affiché à l'utilisateur

Exemple de pattern d'implémentation :

```javascript
const { data, isLoading, isError, error } = useGetUsers();

if (isLoading) return <UserTableSkeleton />;
if (isError) return <ErrorMessage message={error.message} />;

return <UserTable users={data} />;
```

Invalidation du cache : Après chaque mutation réussie (création, modification, suppression), la clé de requête correspondante est invalidée via `queryClient.invalidateQueries(...)` pour forcer un rechargement des données fraîches depuis l'API.

---

2. Guide d'Implémentation des Composants

2.1. Formulaires Dynamiques & Upload de Fichiers

Pour les formulaires incluant des fichiers (photos de profil prestataire, images de bannières, photos de services), le corps de la requête est construit en `FormData` au lieu de JSON.

```javascript
```

```
const handleSubmit = async (formValues) => {
  const formData = new FormData();
  formData.append('title', formValues.title);
  formData.append('description', formValues.description);
  formData.append('price', formValues.price);

  // L'image est ajoutée uniquement si elle est présente
  if (formValues.image instanceof File) {
    formData.append('image', formValues.image);
  }

  // Le client HTTP détecte automatiquement le FormData
  // et omet le Content-Type pour laisser le navigateur le gérer
  await createService(formData);
};
```

2.2. Visualisation des Données

Mapping JSON → Composants :

Les données reçues depuis l'API sont mappées vers les composants de visualisation selon le pattern suivant :

```javascript
// Exemple : graphique des inscriptions via Recharts (interface Admin)
const { data: stats } = useStats();

const chartData = stats?.registrationsByMonth?.map(item => ({
  name: item.month,       // Axe X
  value: item.count,      // Axe Y
})) ?? [];

return (
  <LineChart data={chartData}>
    <XAxis dataKey="name" />
    <YAxis />
    <Line type="monotone" dataKey="value" stroke="#8884d8" />
  </LineChart>
);
```

Composants de visualisation utilisés :

Dans l'interface admin, **Recharts** est utilisé pour les courbes d'inscription, la répartition des utilisateurs (graphiques circulaires) et le résumé des paiements. Dans l'interface client/prestataire, **Leaflet + React Leaflet** affiche une carte interactive des prestataires géolocalisés dans l'écran de recherche (`MapSearch`). **React Google Autocomplete** assiste la saisie d'adresses dans les formulaires de localisation.

2.3. Système de Feedback Utilisateur

États de chargement (Skeletons) : Pendant le chargement initial (`isLoading`), des composants `Skeleton` de `react-loading-skeleton` remplacent les vrais composants pour indiquer visuellement que du contenu est en cours de chargement, sans bloquer l'affichage de la page.

Notifications (Toastify) : Les résultats des actions utilisateur (succès ou échec) sont systématiquement communiqués via des toasts `React Toastify`. Leur déclenchement est centralisé dans les hooks de mutation :

```javascript
const mutation = useMutation({
  mutationFn: createService,
  onSuccess: () => {
    toast.success('Service créé avec succès !');
    queryClient.invalidateQueries(['services']);
  },
  onError: (error) => {
    toast.error(error.message || 'Une erreur est survenue.');
  },
});
```

Animations : Framer Motion est utilisé pour les transitions entre pages et les micro-animations des composants (entrée/sortie de modales). **DotLottie React** sert à afficher des animations Lottie pour les états vides ou de succès.

---

3. Déploiement & Optimisation

3.1. Pipeline de Production

Build avec Vite 7 (Rolldown Vite) :

```bash
# Build de production — génère le dossier /dist optimisé
npm run build

# Prévisualiser le build localement avant déploiement
npm run preview
```

Vite 7 avec le moteur Rolldown produit un bundle optimisé avec tree-shaking, code-splitting automatique et minification. Les assets sont hashés pour la gestion de cache navigateur.

Configuration Nginx en production :

Le fichier de configuration Nginx inclus dans le projet configure le serveur comme suit : il sert les fichiers statiques du dossier `/dist`, applique une mise en cache longue durée sur les assets hashés, et redirige toutes les routes vers `index.html` pour le fonctionnement du routeur SPA React Router.

```nginx
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    # Mise en cache longue durée pour les assets hashés
    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Redirection SPA — toutes les routes vers index.html
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Options de déploiement : Le projet supporte trois cibles de déploiement — **Docker** (via Dockerfile et Nginx), **Vercel** (fichier de configuration inclus pour le déploiement serverless), et tout hébergement statique compatible avec les SPA.

3.2. Performance

Lazy Loading des routes : Les écrans sont chargés à la demande via `React.lazy()` et `Suspense`, réduisant le bundle initial et accélérant le premier affichage.

```javascript
const SearchScreen = React.lazy(() => import('./screens/SearchScreen'));
const ProviderScreen = React.lazy(() => import('./screens/ProviderScreen'));

// Dans le routeur :
<Suspense fallback={<PageSkeleton />}>
  <Routes>
    <Route path="/search" element={<SearchScreen />} />
    <Route path="/provider" element={<ProviderScreen />} />
  </Routes>
</Suspense>
```

Mise en cache côté client avec React Query : React Query maintient un cache en mémoire des réponses API. Le `staleTime` peut être configuré par hook pour contrôler la durée pendant laquelle les données sont considérées

fraîches et ne déclenchent pas de re-fetch. Pour les données peu volatiles (liste des plans d'abonnement, catégories de services), un `staleTime` long évite des appels réseau redondants.

Optimisation des images : Les images prestataires étant hébergées sur Cloudinary, les URLs sont générées avec des paramètres de transformation (redimensionnement, format WebP, qualité adaptative) selon le contexte d'affichage, réduisant le poids des ressources transmises.

CONCLUSION

Maintenance et évolutivité (SaaS)

AELI Services est conçu avec une architecture modulaire orientée vers l'évolutivité SaaS. Quelques axes directeurs pour la maintenance :

Versionning de l'API : Lors de changements de contrats d'interface incompatibles, il est recommandé d'introduire un préfixe de version (`/api/v2/...`) plutôt que de modifier les endpoints existants, afin de ne pas rompre les clients en production.

Migrations de base de données : Toute modification du schéma doit passer par une migration Sequelize versionnée et être testée en environnement de staging avant la mise en production. Les migrations doivent être réversibles (`up` et `down`).

Tests de non-régression : Le projet compte 596 tests (82% de couverture de lignes). Tout ajout de fonctionnalité doit s'accompagner de tests unitaires et d'intégration correspondants. La couverture des routes admin (actuellement ~45%) est identifiée comme axe d'amélioration prioritaire.

Monitoring : En production, il est recommandé de coupler Winston à un agrégateur centralisé (Datadog, Logtail) et d'instrumenter l'API avec un outil APM pour suivre les performances des endpoints et anticiper les goulots d'étranglement.

Internationalisation : Le backend supporte déjà le français et l'anglais via `i18n`. L'ajout d'une nouvelle langue requiert uniquement l'ajout d'un fichier de traduction dans `src/locales/`.

Lexique Technique

Terme	Définition
JWT (JSON Web Token)	Standard ouvert de transmission sécurisée d'informations entre parties sous forme de token signé.
Access Token	Token JWT de courte durée utilisé pour authentifier les requêtes API.
Refresh Token	Token JWT de longue durée utilisé uniquement pour obtenir un nouvel Access Token expiré.

Terme	Définition
Silent Refresh	Mécanisme de renouvellement automatique du token d'accès sans interaction utilisateur.
OTP (One-Time Password)	Code à usage unique envoyé par email pour valider une identité.
ORM (Object-Relational Mapper)	Couche d'abstraction entre le code et la base de données (ici : Sequelize).
Migration	Script de modification versionnée du schéma de base de données.
Rate Limiting	Mécanisme de limitation du nombre de requêtes par IP sur une période donnée.
FormData	Interface JavaScript pour construire des requêtes HTTP multipart/form-data (upload de fichiers).
SPA (Single Page Application)	Application web dont toute la navigation est gérée côté client par un routeur JavaScript.
SSR	Rendu côté serveur (non utilisé ici — l'architecture est 100% SPA).
React Query	Bibliothèque de gestion du cache et des états de données asynchrones dans React.
Hook	Fonction React réutilisable encapsulant de la logique avec accès aux états et au cycle de vie.
Skeleton	Composant de chargement affichant une silhouette du contenu en cours de récupération.
Toast	Notification non-modale s'affichant temporairement pour informer l'utilisateur d'un résultat.
Bull	Bibliothèque Node.js de gestion de files de tâches asynchrones, s'appuyant sur Redis.
Cloudinary	Service cloud de stockage, transformation et livraison d'images et vidéos.
NotchPay / CinetPay	Passerelles de paiement mobile money utilisées pour les transactions sur la plateforme.
AES-256	Standard de chiffrement symétrique utilisé pour protéger les données personnelles en base.
bcrypt	Fonction de hachage adaptative utilisée pour le stockage sécurisé des mots de passe.
CORS	Mécanisme de sécurité navigateur contrôlant les requêtes cross-origine.
Helmet	Middleware Express appliquant des en-têtes HTTP de sécurité.
Lazy Loading	Technique de chargement différé des ressources ou composants à la demande.

Terme	Définition
Tree-shaking	Optimisation du bundler éliminant le code JavaScript non utilisé du bundle final.
Webhook	Callback HTTP déclenché par un service tiers (ici : confirmation de paiement).